Session 13 Homework — A Second Real Filter, and an Honest Conclusion About the Index You Already Have

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, Issues support filtering by `projectId` and `status` together, backed by a real, measured compound index. This homework adds a third, different kind of search — text search by `title` — and asks you to reason honestly about whether your existing indexes already help it, or not.

Goal

`GET /issues` gains partial, case-insensitive `title` filtering, using the identical pattern already used twice this course. You then write a precise, honest conclusion about exactly which part (if any) of your existing `{ projectId, status }` index helps this new search — and which part it does not.

What to build

Add `title` filtering to Issues (CORE), using only syntax already introduced during this session; finish Swagger response documentation on every guarded Issues route (STRETCH); and write a short, specific comparison directly on the `Issue` schema (CHALLENGE).

CORE (required)

Add partial, case-insensitive `title` filtering to Issues:

- `@IsOptional()` / `@IsString()` on a new `title?: string` field on the query DTO — the identical shape already used for Projects' own `name` and Workspaces' own `name`.
- In `IssuesService`'s own `findAll()`, filter using `$regex` / `$options: 'i'` — the identical pattern already used twice.
- Directly above that new filter, write a short, honest comment: does your existing `{ projectId, status }` compound index help this new `title` search? Be precise — if it helps in SOME cases but not others, say exactly which.

STRETCH (required)

Using only syntax already introduced during this session, finish response documentation on Issues:

- Add `@ApiResponse` for both the real success status and the real `403` on every already-guarded Issues route (`create`, `update`, `remove`) — the identical standard already held for Projects and Workspaces in Session 12's own homework.

- Never expose the actual internal key value anywhere in Swagger — only that the header is required.

CHALLENGE (required)

Write a short, direct comparison as a comment block on the `Issue` schema itself, directly above the class:

Why are Issues referenced from Projects, while Comments can later be embedded inside Issues? Name the ONE real deciding factor — not a list of surface differences between "a reference" and "an embed."

Rules

Allowed: everything class already used — `@IsString`, `$regex` / `$options: 'i'`, `@ApiResponse`, `explain("executionStats")`. No new syntax this homework introduces anything beyond what live CORE and teamwork already built. **Not yet:** no JWT, no users, no roles, no custom decorators, no interceptors, no Mongoose `ObjectId` / `ref` / `populate()`, no aggregation pipelines, no pagination.

Test it

BASH

```
curl -s 'localhost:3000/issues?title=login'
```

Then, in `mongosh`, check what your own existing indexes actually do for this new query shape — run `explain("executionStats")` on a `title`-only search, and separately on a search that combines `title` with `projectId`, and compare what you see before writing your CORE conclusion.

Done when

- `GET /issues?title=<text>` returns a real, correctly-filtered, case-insensitive partial match.
- Your written conclusion about index coverage is precise — it says exactly which part of your existing index helps (if any), not a blanket yes-or-no rule.
- Every guarded Issues route documents both its real success status and its `403` in Swagger, with no exposed key value anywhere.
- Your CHALLENGE comparison names the one real deciding factor (bounded vs. unbounded growth, read-together vs. read-independently), not just a definition of "reference" and "embed."

Working time

About 60-90 minutes — CORE is one known filter pattern plus a short, careful written conclusion (verify it with `explain()` if you're not sure, don't guess), STRETCH is documentation completeness across already-written code, and CHALLENGE is a short written comparison, not new code.